

SEIP Assignment

Luca Janek Schröder (t8250344)

[Forked Repo](#)

Successful CI/CD Pipeline Execution

The screenshot shows the GitHub Actions interface for a repository named 'seip_assignment_1_2026'. The workflow 'feat: containerize, add CI/CD pipeline, and deploy to Kubernetes #3' is displayed, showing a successful execution of the 'build-and-push' job. The job summary indicates it succeeded 3 minutes ago. The job steps are listed as follows:

Step	Duration
Set up job	2s
Run actions/checkout@v4	0s
Log in to GHCR	1s
Build and push	1.7s
Post Build and push	0s
Post Log in to GHCR	1s
Post Run actions/checkout@v4	0s
Complete job	0s

Three Replicas Running in Default Namespace

```
PS C:\Users\lschr\OneDrive\Desktop\Business Analytics & Econometrics\Semester Abroad\Software engineering\lab_assignment\seip_assignment_1_2026> kubectl get all -n default
```

NAME	READY	STATUS	RESTARTS	AGE
pod/echo-api-59b46b7959-lxggm	1/1	Running	0	8m
pod/echo-api-59b46b7959-tm95b	1/1	Running	0	7m52s
pod/echo-api-59b46b7959-tsz5c	1/1	Running	0	8m14s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/echo-api-service	ClusterIP	10.106.196.45	<none>	80/TCP	55m
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	105m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/echo-api	3/3	3	3	55m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/echo-api-59b46b7959	3	3	3	8m14s

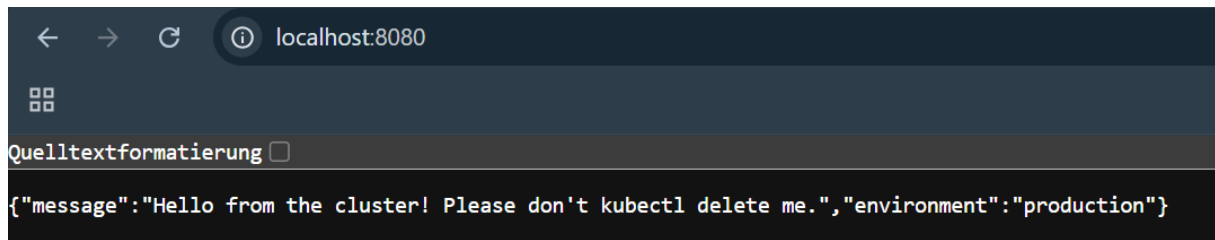
ConfigMap and Secret Applied to Cluster

```
PS C:\Users\lschr\OneDrive\Desktop\Business Analytics & Econometrics\Semester Abroad\Software engineering\lab_assignment\seip_assignment_1_2026> kubectl get configmap,secret
```

NAME	DATA	AGE
configmap/echo-api-config	2	57m
configmap/kube-root-ca.crt	1	106m

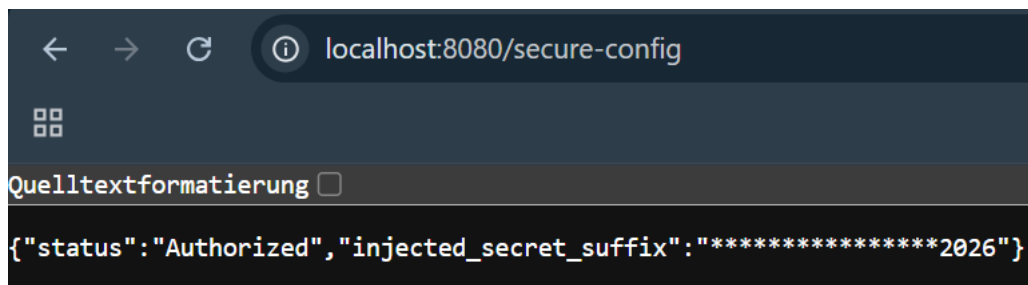
NAME	TYPE	DATA	AGE
secret/echo-api-secret	Opaque	1	57m

Root Endpoint Returning Welcome Message



```
localhost:8080
{"message": "Hello from the cluster! Please don't kubectl delete me.", "environment": "production"}
```

Secure Config Endpoint with Masked API Key



```
localhost:8080/secure-config
{"status": "Authorized", "injected_secret_suffix": "*****2026"}
```

AI Integration

I used Claude for most of this assignment. The pattern was the same throughout: describe what I was trying to configure, get a draft manifest or snippet, run it, and come back with the actual error output. For smaller completions inside the Dockerfile and the GitHub Actions YAML, GitHub Copilot was faster.

What Was Actually Useful

- Explaining the difference between containerPort in a Deployment and port in a Service, which I kept conflating early on
- The base64 encoding command with the -n flag. I had used plain echo first and encoded a trailing newline into the secret, which caused the app to read a corrupted key. The AI explained why -n matters, not just what to type
- Walking through why GITHUB_TOKEN needs package write permissions for GHCR specifically
- A working liveness/readiness probe block in about 30 seconds, which would have taken me significantly longer reading the Kubernetes docs

Where It Fell Short

The main issue was Minikube. The AI suggested `kubectl port-forward` by default, but that did not work with my setup because it depends on which Minikube driver you are running. I had not specified that upfront, so the suggestion was reasonable but wrong for my case. I had to check the Minikube docs myself to figure out that `minikube service` was the right call for my driver.

The other issue was GitHub Actions versioning. The `docker/build-push-action` snippet used a `with:` key that no longer existed in the current version of that action. The workflow failed with a clear error so I caught it, but the AI did not flag that its knowledge of that action's API might be outdated.

What I Would Build Next

Three things stand out as the most obvious next steps for this specific setup:

- **Ingress Controller:** the port-forwarding I used for testing is not a real solution. An NGINX Ingress would make the service reachable by hostname and survives pod restarts, which port-forward does not
- **Image scanning in CI:** the pipeline currently pushes whatever the Dockerfile builds without checking for known CVEs in the base image. Adding Trivy before the push step would catch a vulnerable `node:18-alpine` before it reaches the registry
- **GitOps with ArgoCD:** right now the CI pipeline applies manifests directly to the cluster. That means the cluster state can drift from what is in git without anyone noticing. ArgoCD would make git the single source of truth and flag any drift automatically